

23. 合并 K 个升序链表

合并 K 个升序链表

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Hard (61.64%)	3005	0	-	-	0

- **Tags**
- **Companies**

给你一个链表数组，每个链表都已经按升序排列。

请你将所有链表合并到一个升序链表中，返回合并后的链表。

示例 1:

输入: `lists = [[1,4,5],[1,3,4],[2,6]]`

输出: `[1,1,2,3,4,4,5,6]`

解释: 链表数组如下:

```
[
  1->4->5,
  1->3->4,
  2->6
]
```

将它们合并到一个有序链表中得到。
1->1->2->3->4->4->5->6

示例 2:

输入: `lists = []`

输出: `[]`

示例 3:

输入: `lists = [[]]`

输出: `[]`

提示:

- `k == lists.length`
- `0 <= k <= 10^4`
- `0 <= lists[i].length <= 500`
- `10^4 <= lists[i][j] <= 10^4`
- `lists[i]` 按 升序 排列
- `lists[i].length` 的总和不超过 `10^4`

[Discussion](#) | [Solution](#)

LeetCode 23: 合并 K 个升序链表（详细解析）

1. 问题理解

目标：将 k 个升序排列的链表合并成一个单一的升序链表。

输入：一个包含 k 个链表头节点的数组 `ListNode[] lists`。数组中的每个链表都已按升序排序。

输出：合并后的单个升序链表的头节点。

示例：

- 输入: `lists = [[1,4,5],[1,3,4],[2,6]]`
- 输出: `[1,1,2,3,4,4,5,6]`

关键点：

- 输入是多个已排序的链表。
- 需要将所有链表的节点合并到一个新的有序链表中。
- 需要考虑输入数组为空、数组中包含空链表等边界情况。

2. 思路分析

如何高效地从 k 个链表的当前头节点中找到最小的那个节点，并将其添加到结果链表中？

思路一：逐一合并（类似 **LeetCode 21** 的多次应用）

想法：

1. 从结果为空开始，或者以第一个链表作为初始结果。
2. 遍历数组中的剩余链表，每次都当前结果链表与下一个链表进行合并（使用合并两个有序链表的逻辑）。
3. 重复此过程，直到所有链表都合并完毕。

复杂度：

- 假设总共有 N 个节点， k 个链表。
- 合并两个链表的时间复杂度是 $O(m+n)$ 。
- 如果每次合并都创建一个新链表，或者即使原地合并，合并第 i 个链表时，结果链表的长度大约是前 $i-1$ 个链表节点总数。
- 粗略估计，时间复杂度约为 $O(kN)$ 。每次合并操作都需要遍历当前合并结果和下一个链表。
- 空间复杂度： $O(1)$ （如果原地修改指针）或 $O(N)$ （如果创建新节点）。辅助空间为 $O(1)$ 。

缺点：当 k 很大时，效率不高，因为较早合并的链表会被反复遍历。

思路二：使用优先队列（最小堆）

想法：利用最小堆（PriorityQueue）来高效地找到 k 个链表当前头节点中的最小值。

步骤：

1. 创建一个最小堆，用于存储 `ListNode`，并根据节点的 `val` 进行排序。
2. 将 k 个链表的非空头节点全部加入最小堆。
3. 创建一个哑节点 `dummy` 和一个尾指针 `tail` 用于构建结果链表。
4. 循环：当最小堆不为空时：
 - a. 从堆中取出（`poll()`）值最小的节点 `minNode`。
 - b. 将 `minNode` 连接到结果链表的末尾（`tail.next = minNode`）。
 - c. 移动 `tail` 指针（`tail = tail.next`）。
 - d. 如果 `minNode` 还有下一个节点（`minNode.next != null`），则将其下一个节点加入（`offer()`）最小堆。
5. 循环结束后，返回 `dummy.next`。

复杂度：

- N 是所有链表的总节点数， k 是链表的数量。
- 将 k 个头节点加入堆： $O(k \log k)$ 。
- 每次从堆中取出最小节点： $O(\log k)$ 。
- 每次向堆中添加下一个节点： $O(\log k)$ 。
- 总共有 N 个节点需要处理（每个节点进堆一次，出堆一次）。
- 总时间复杂度： $O(N \log k)$ 。
- 空间复杂度： $O(k)$ ，堆中最多存储 k 个节点（每个链表一个）。

优点：时间复杂度更优，尤其当 k 较大时。

思路三：分治法（归并排序思想）

想法：模仿归并排序的过程，两两合并链表，不断减少链表的数量，直到只剩下一个合并后的链表。

步骤（递归）：

1. 定义一个函数 `mergeRange(lists, start, end)`，表示合并 `lists` 数组中从索引 `start` 到 `end` 的链表。
2. 基本情况：
 - 如果 `start > end`，返回 `null`。
 - 如果 `start == end`，返回 `lists[start]`。
3. 递归步骤：
 - 找到中间点 `mid = start + (end - start) / 2`。
 - 递归合并左半部分：`leftMerged = mergeRange(lists, start, mid)`。
 - 递归合并右半部分：`rightMerged = mergeRange(lists, mid + 1, end)`。

- 合并 `leftMerged` 和 `rightMerged` (使用 LeetCode 21 的逻辑), 并返回结果。
4. 初始调用 `mergeRange(lists, 0, lists.length - 1)`。

步骤 (迭代):

1. 使用一个变量 `interval` 控制合并的步长, 初始为 1。
2. 当 `interval < k` 时:
 - 遍历 `lists` 数组, 步长为 `2 * interval`。对于每个 `i`, 合并 `lists[i]` 和 `lists[i + interval]` (如果 `i + interval` 存在), 并将结果存回 `lists[i]`。
 - `interval *= 2`。
3. 最终结果在 `lists[0]`。

复杂度:

- 每次合并两个链表的时间与其总长度成正比。每一轮合并 (将链表数量减半) 都需要处理所有 N 个节点。
- 总共有 $O(\log k)$ 轮合并。
- 总时间复杂度: $O(N \log k)$ 。
- 空间复杂度:
 - 递归法: $O(\log k)$, 递归调用栈的深度。
 - 迭代法: $O(1)$, 原地修改 `lists` 数组。

比较:

- 优先队列和分治法的时间复杂度都是 $O(N \log k)$, 优于逐一合并的 $O(kN)$ 。
- 优先队列需要 $O(k)$ 的额外空间。
- 分治法的迭代版本空间复杂度最优, 为 $O(1)$ 。

3. 代码实现 (优先队列法)

```
-sorted-lists.java
import java.util.PriorityQueue;
import java.util.Comparator;

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    /**
     * 合并 K 个升序链表 - 优先队列 (最小堆) 法
```

```

    * @param lists 包含 k 个链表头节点的数组
    * @return 合并后的升序链表的头节点
    */
    public ListNode mergeKLists(ListNode[] lists) {
        // 处理边界情况
        if (lists == null || lists.length == 0) {
            return null;
        }

        // 创建最小堆，并定义比较器
        // PriorityQueue<ListNode> minHeap = new PriorityQueue<>(Comparator.comparingInt((no
        // 或者使用 lambda 表达式
        PriorityQueue<ListNode> minHeap = new PriorityQueue<>((a, b) -> a.val - b.val);

        // 将每个链表的非空头节点加入最小堆
        for (ListNode head : lists) {
            if (head != null) {
                minHeap.offer(head);
            }
        }

        // 创建哑节点和尾指针
        ListNode dummy = new ListNode(0);
        ListNode tail = dummy;

        // 循环处理堆中的节点
        while (!minHeap.isEmpty()) {
            // 取出堆顶（最小）节点
            ListNode minNode = minHeap.poll();

            // 将最小节点连接到结果链表
            tail.next = minNode;
            tail = tail.next; // 移动尾指针

            // 如果最小节点还有下一个节点，将其加入堆
            if (minNode.next != null) {
                minHeap.offer(minNode.next);
            }
        }

        // 返回哑节点的下一个节点
        return dummy.next;
    }
}

```

4. 代码解析 (优先队列法)

- 边界处理: 检查输入数组 `lists` 是否为 `null` 或空。
- 创建最小堆: `PriorityQueue<ListNode> minHeap = new PriorityQueue<>((a, b) -> a.val - b.val);` 创建一个优先队列, 并提供一个比较器, 使得队列按照节点的 `val` 值升序排列 (值小的优先级高)。
- 初始化堆: 遍历 `lists` 数组, 将每个链表的第一个非空节点加入堆中。
- 哑节点和尾指针: `dummy` 用于方便地构建新链表, `tail` 始终指向新链表的最后一个节点。
- 主循环 `while (!minHeap.isEmpty())`: 只要堆中还有节点, 就说明还有元素需要合并。
- `minHeap.poll()`: 取出并移除堆顶元素, 这一定是当前所有链表头节点中值最小的那个。
- 连接节点: 将取出的 `minNode` 连接到 `tail` 后面, 并更新 `tail`。
- 添加下一节点: `if (minNode.next != null) { minHeap.offer(minNode.next); }` 这是关键一步。将被取出节点的下一个节点 (如果存在) 加入堆中, 以便后续比较。
- 返回结果: `dummy.next` 是合并后链表的真正头节点。

5. 如何在面试中讲解

1. 理解问题:
 - “目标是将 k 个已排序的链表合并成一个大的排序链表。”
2. 讨论不同方法及其复杂度:
 - 逐一合并: “可以像合并两个链表一样, 循环 $k-1$ 次, 每次合并当前结果和下一个链表。时间复杂度 $O(kN)$, 空间 $O(1)$ 。当 k 很大时效率不高。”
 - 优先队列 (最小堆): “一个更优的方法是使用最小堆。将 k 个链表的头节点放入堆中。每次取出堆顶 (最小节点), 加入结果链表, 然后将被取出的节点的下一个节点 (如果存在) 放入堆中。时间复杂度 $O(N \log k)$, 空间 $O(k)$ 。”
 - 分治法: “还可以用分治的思想, 类似归并排序。两两合并链表, 不断减少链表数量, 直到剩下一个。时间复杂度 $O(N \log k)$ 。递归实现空间 $O(\log k)$, 迭代实现空间 $O(1)$ 。”
3. 选择并详细讲解一种方法 (以优先队列为例):
 - “优先队列 (最小堆) 是一个常用且高效的选择。”
 - “首先, 创建一个能按节点值排序的最小堆。”
 - “将所有输入链表的非空头节点加入堆。”
 - “然后, 创建一个哑节点 `dummy` 和尾指针 `tail` 来构建结果链表。”
 - “当堆不为空时, 重复以下操作:”
 - “从堆中取出最小节点 `minNode`。”
 - “将 `minNode` 追加到 `tail` 后面, 并更新 `tail`。”
 - “如果 `minNode` 有下一个节点, 将其下一个节点加入堆。”
 - “最后返回 `dummy.next`。”
4. 分析所选方法的复杂度:
 - “时间复杂度是 $O(N \log k)$, 因为总共有 N 个节点, 每个节点最多进堆一次、出堆一次, 堆操作是 $O(\log k)$ 。”

- “空间复杂度是 $O(k)$ ，因为堆的大小最多为 k 。”
5. 提及其他方法的优劣（可选）：
- “如果空间复杂度是首要考虑因素，迭代的分治法可以达到 $O(1)$ 空间，但实现可能稍复杂。”

优先队列法思路清晰，代码相对容易实现，是面试中常见的解决方案。

找到具有 1 个许可证类型的类似代码